

**Automatización de la configuración de almacenamiento en máquinas virtuales  
GNU/Linux desde el host con Oracle VM VirtualBox**

**Estudiantes:**

Jorge Iván Acosta Aristizábal

[Compañero 2]



**Universidad del Quindío**  
**Facultad Ingeniería**  
**Ingeniería de Sistemas y Computación**  
Computación en la Nube  
Armenia, Quindío  
Octubre del 2026

# Índice

|                                                               |           |
|---------------------------------------------------------------|-----------|
| <b>Introducción</b>                                           | <b>4</b>  |
| <b>1. Marco Teórico y Tecnologías</b>                         | <b>5</b>  |
| 1.1. Virtualización y Oracle VM VirtualBox . . . . .          | 5         |
| 1.2. Guest Additions . . . . .                                | 6         |
| 1.3. Comunicación Host–Guest . . . . .                        | 6         |
| 1.4. Gestión de Almacenamiento en GNU/Linux . . . . .         | 7         |
| 1.5. Automatización con Bash . . . . .                        | 8         |
| 1.6. Testing de Scripts . . . . .                             | 9         |
| <b>2. Diseño de la Solución</b>                               | <b>9</b>  |
| 2.1. Escenario de Aplicación . . . . .                        | 9         |
| 2.2. Arquitectura y Comunicación entre Host y Guest . . . . . | 11        |
| 2.3. Algoritmo de Automatización (Idempotente) . . . . .      | 13        |
| 2.4. Interfaz de la Solución . . . . .                        | 16        |
| 2.5. Sintaxis del Archivo Declarativo . . . . .               | 18        |
| 2.6. Instalación y Sistema de Construcción . . . . .          | 21        |
| <b>3. Implementación</b>                                      | <b>22</b> |
| 3.1. Estructura del Código . . . . .                          | 22        |
| 3.2. Script / Programa Principal . . . . .                    | 25        |
| 3.3. Parametrización para Múltiples VMs . . . . .             | 35        |
| 3.4. Manejo de Errores y Casos Edge . . . . .                 | 36        |
| 3.5. Verificación Estática y Pruebas Unitarias . . . . .      | 38        |
| 3.6. Evidencias de las Pruebas de Funcionamiento . . . . .    | 39        |
| 3.7. Tiempo de Ejecución . . . . .                            | 41        |

|                                                                   |           |
|-------------------------------------------------------------------|-----------|
| <b>4. Conclusiones</b>                                            | <b>42</b> |
| 4.1. Logros Alcanzados . . . . .                                  | 42        |
| 4.2. Cumplimiento de Requisitos del Parcial . . . . .             | 43        |
| 4.3. Problemas Encontrados y su Solución . . . . .                | 44        |
| 4.4. Herramientas de Inteligencia Artificial Utilizadas . . . . . | 45        |
| 4.5. Limitaciones Identificadas . . . . .                         | 46        |
| 4.6. Trabajo Futuro . . . . .                                     | 46        |

En prácticas previas de la asignatura se realizó manualmente la preparación de un disco adicional para ser utilizado como almacenamiento de datos en una máquina virtual GNU/Linux. Dicha actividad implicó ejecutar secuencialmente los procedimientos de particionado, creación del sistema de archivos (ext4/xfs), montaje y configuración del montaje automático mediante `/etc/fstab` para garantizar persistencia (Nemeth et al., 2017). Si bien estos pasos son fundamentales para la administración de almacenamiento, su ejecución manual resulta repetitiva, propensa a errores humanos y no escala cuando una organización debe aprovisionar almacenamiento en múltiples máquinas virtuales.

El presente trabajo aborda la automatización completa de este proceso desde el sistema operativo *host*, aprovechando las capacidades de Oracle VM VirtualBox. La solución permite que el *host* inicie y prepare una *VM*, establezca comunicación con el sistema operativo invitado mediante las Guest Additions y ejecute automáticamente las acciones necesarias para que el nuevo almacenamiento quede disponible sin intervención manual. Los requisitos fundamentales son la ejecución desatendida, sin ingreso de contraseñas, comandos interactivos ni pulsaciones de tecla; la reproducibilidad, con el mismo resultado en cada ejecución; la reutilización, aplicable a múltiples *VMs* con parámetros configurables; y la idempotencia, con re-ejecución segura sobre *VMs* ya configuradas sin pérdida de datos ni operaciones destructivas (Wikipedia Contributors, 2024).

La comunicación *host-VM* se realiza a través de la Guest Control API (`VBoxManage guestcontrol`), mecanismo nativo de VirtualBox que no requiere configuración de red adicional más allá del modo puente (*bridged networking*) para la obtención de la IP de la *VM* (Oracle Corporation, 2024c, 2024e). La automatización se implementa en Bash para *host* Linux/macOS, siguiendo prácticas robustas de scripting (GreyCat, 2024; Holen, 2024) y validada con el framework de testing BATS (Bats Contributors, 2024). De este modo, el aprovisionamiento deja de depender de pasos manuales y se convierte en un procedimiento desatendido y verificable, cuyos resultados son consistentes entre ejecuciones y cuya repetición no compromete al sistema invitado.

# 1. Marco Teórico y Tecnologías

La solución descrita en este documento se inscribe en la confluencia de tres áreas complementarias de la administración de sistemas. La virtualización permite ejecutar sistemas operativos completos como *VM* aisladas sobre un mismo equipo físico y expone los recursos, como discos e interfaces de red, que el *host* puede administrar de forma programática (Oracle Corporation, 2024d, 2024e). La administración de almacenamiento en GNU/Linux, por su parte, define el procedimiento que se busca automatizar: dejar un disco disponible y persistente en el sistema invitado (Nemeth et al., 2017). Por último, la automatización mediante scripts convierte ese procedimiento manual en una operación programada, reproducible e idempotente, es decir, capaz de repetirse sin alterar el resultado ni comprometer los datos ya existentes (Wikipedia Contributors, 2024). Estos tres frentes convergen cuando una organización debe preparar almacenamiento en múltiples *VM* sin intervención manual, que es precisamente el escenario abordado a continuación.

## 1.1. Virtualización y Oracle VM VirtualBox

Oracle VM VirtualBox es un hipervisor de tipo 2 (alojado, *hosted*), es decir, una aplicación que se instala sobre el sistema operativo del *host* y permite ejecutar múltiples sistemas invitados como *VM* independientes sin requerir modificaciones al sistema anfitrión (Oracle Corporation, 2024d). Este modelo resulta apropiado para entornos de trabajo y educativos, donde la instalación sencilla y la gestión centralizada desde una única interfaz son requisitos frecuentes.

Entre las capacidades de VirtualBox relevantes para este trabajo se encuentran la gestión de discos virtuales en formatos como `.vdi` y `.vmdk`, que permiten adjuntar almacenamiento adicional a una *VM* sin intervención sobre hardware físico, y la configuración de red en modo puente (*bridged networking*), en la cual la interfaz de la *VM* se asocia a una interfaz física del *host* y obtiene una dirección IP propia dentro de la red local

(Oracle Corporation, 2024e). Asimismo, VirtualBox expone una interfaz completa de línea de comandos, `VBoxManage`, mediante la cual es posible crear, iniciar, detener y consultar *VM*, administrar discos y snapshots y consultar o modificar propiedades; dicho comando constituye el punto de entrada para toda la automatización descrita en este documento (Oracle Corporation, 2024d).

## 1.2. Guest Additions

Las Guest Additions son un conjunto de controladores de dispositivo y servicios que se instalan dentro del sistema operativo invitado y que mejoran la integración y el rendimiento entre el *host* y la *VM*: gráficos acelerados, portapapeles compartido, redimensionamiento dinámico de la ventana y sincronización de hora, entre otros (Oracle Corporation, 2024b).

Su componente de mayor relevancia para este trabajo es el servicio de Guest Control, que habilita desde el *host* el comando `VBoxManage guestcontrol`. A través de este mecanismo es posible ejecutar programas en el invitado, copiar archivos, crear directorios y gestionar sesiones, autenticándose con un usuario y contraseña del propio sistema invitado (Oracle Corporation, 2024a, 2024c). Para que todo ello funcione, las Guest Additions deben estar instaladas en la *VM* y el servicio `VBoxService` debe encontrarse en ejecución; de ello depende la ejecución desatendida que la solución propone (Oracle Corporation, 2024b).

## 1.3. Comunicación Host–Guest

Existen varios mecanismos para ejecutar comandos en el invitado desde el *host*, cada uno con un distinto nivel de dependencia respecto de la configuración de red.

La Guest Control API (`VBoxManage guestcontrol`) permite ejecutar procesos, transferir archivos y crear directorios dentro de la *VM* sin que el invitado exponga servicios de red adicionales, pues la comunicación la transporta el propio hipervisor por medio de las Guest

Additions (Oracle Corporation, 2024a, 2024c). Como alternativa, SSH con autenticación por clave ofrece un canal estandarizado y ampliamente disponible en sistemas GNU/Linux, siempre que la VM ejecute un servidor SSH y exista conectividad de red; su configuración comprende el intercambio de llaves y opciones de cliente como `StrictHostKeyChecking`, `UserKnownHostsFile` y `ConnectTimeout` (OpenSSH Project, 2024a, 2024b). Finalmente, las carpetas compartidas facilitan la transferencia de archivos entre *host* y *VM*, pero no sustituyen la ejecución remota de comandos (Oracle Corporation, 2024a).

La solución adopta la Guest Control API por ser nativa de VirtualBox y por reducir la superficie de configuración: no exige instalar ni habilitar servicios de red en el invitado ni gestionar llaves SSH. El modo puente se configura igualmente para que la VM obtenga una dirección IP propia en la red local, lo que permite identificarla y verificar su estado durante la ejecución de la automatización (Oracle Corporation, 2024c, 2024e).

## 1.4. Gestión de Almacenamiento en GNU/Linux

En GNU/Linux el almacenamiento se modela mediante dispositivos de bloques, cuyos nombres dependen del controlador: `/dev/sdX` para discos SCSI, SATA y USB, `/dev/vdX` para discos virtuales y `/dev/nvme*` para unidades NVMe (The Linux Kernel Archives, 2024). Sobre un disco se crean particiones con herramientas como `fdisk` o `parted`; la tabla puede ser MBR o GPT, y esta última es más moderna y recomendada (Nemeth et al., 2017).

Cada partición requiere un sistema de archivos, por ejemplo `ext4` o `xfs`, que se crea con la utilidad correspondiente (`mkfs.ext4`, `mkfs.xfs`) (Nemeth et al., 2017). Dicho sistema de archivos se hace accesible en un punto de montaje de la jerarquía de directorios, para la cual el estándar de jerarquía del sistema de archivos reserva rutas como `/mnt` y `/media` (Linux Foundation, 2015). La operación `mount` asocia la partición con su punto de montaje, pero ese vínculo no persiste tras un reinicio salvo que se declare en `/etc/fstab` o en una unidad de montaje de `systemd` (Nemeth et al., 2017; `systemd` Developers, 2024).

Para que la identificación sea estable, las entradas de `/etc/fstab` deben referenciar

el sistema de archivos mediante su UUID, obtenido con `blkid`, y no mediante `/dev/sdX`, ya que la letra asignada a un dispositivo puede cambiar entre reinicios o al conectar otro disco (Ubuntu Community, 2023). Una vez aplicada la configuración, su verificación se realiza con herramientas como `lsblk`, `findmnt` y `df -h` (Nemeth et al., 2017).

## 1.5. Automatización con Bash

La automatización de la preparación de almacenamiento exige un lenguaje disponible de forma nativa en el *host*, con acceso directo a las utilidades de línea de comandos de GNU/Linux y con la madurez necesaria para construir scripts robustos. La solución se implementa íntegramente en Bash, intérprete de comandos presente de forma predeterminada en los sistemas GNU/Linux y macOS y heredero del lenguaje de comandos definido por la norma POSIX, lo que favorece su portabilidad dentro de ese entorno (GreyCat, 2024; IEEE and The Open Group, 2018). Con ello, el alcance queda concentrado en una única implementación en Bash, en lugar de mantener además una versión paralela para otros sistemas operativos.

La robustez de los scripts se apoya en prácticas consolidadas del lenguaje: activar el modo estricto de ejecución (`set -euo pipefail`), entrecomillar adecuadamente las expansiones, emplear arrays y funciones en lugar de cadenas de comandos y verificar explícitamente los códigos de salida de cada instrucción (GreyCat, 2024). Sobre esa base, el algoritmo incorpora el principio de idempotencia: una operación idempotente produce el mismo resultado sin efectos secundarios aunque se aplique repetidamente, de modo que re-ejecutar la automatización sobre una VM ya configurada no reformatea discos ni duplica entradas en `/etc/fstab` (Hochstein & Zadka, 2023; Wikipedia Contributors, 2024).

La calidad de ese código se verifica con dos herramientas complementarias. ShellCheck es un analizador estático que examina los scripts antes de ejecutarlos y detecta errores comunes y malas prácticas, como expansiones sin entrecomillar, comandos inexistentes o manejo deficiente de la salida, sin necesidad de correr el programa (Holen, 2024). BATS



(*Bash Automated Testing System*), en cambio, ejecuta pruebas automatizadas sobre el comportamiento del script, incluida la re-ejecución idempotente sobre una *VM* ya preparada (Bats Contributors, 2024). De este modo, los defectos se detectan en etapa de desarrollo y no durante la configuración del sistema invitado.

## **1.6. Testing de Scripts**

La verificación de los scripts combina análisis estático y pruebas funcionales. El análisis estático se realiza con ShellCheck sobre el código fuente, lo que permite identificar defectos potenciales en etapa de desarrollo y reduce el costo de corrección respecto a los fallos detectados durante la ejecución sobre una *VM* (Holen, 2024). Dicha verificación se ejecuta de forma previa a cualquier prueba funcional, de manera que solo los scripts que superan el linter avanzan a la siguiente etapa (GreyCat, 2024).

Las pruebas funcionales se implementan con BATS, framework diseñado específicamente para scripts Bash, que organiza cada caso mediante bloques de configuración y limpieza (*setup/teardown*) y aserciones, y que produce salida en formato TAP, compatible con herramientas de integración continua (Bats Contributors, 2024). En este trabajo, BATS se emplea para verificar los comportamientos críticos de la automatización: la preparación correcta de un disco nuevo, la detección de una configuración existente y, en particular, la re-ejecución sobre una *VM* ya preparada, la cual debe finalizar con éxito sin modificar los datos persistidos (Wikipedia Contributors, 2024).

## **2. Diseño de la Solución**

### **2.1. Escenario de Aplicación**

El escenario de trabajo lo integra un equipo *host* que ejecuta GNU/Linux con Oracle VM VirtualBox 7.2 y con la solución en Bash instalada, junto con dos máquinas virtuales

GNU/Linux llamadas VM1 y VM2, que comparten el mismo sistema invitado, Debian 13 (trixie), pero se configuran con parámetros distintos. El *host* participa en la red local a través de una de sus interfaces físicas. En este escenario esa interfaz es `enp2s0`, aunque el nombre depende de la topología de cada equipo. Cada VM posee una interfaz de red en modo puente que la conecta a la red local `192.168.1.0/24` y que en el invitado se denota `enp0s3`. Tal designación resulta del controlador y del orden de los adaptadores, de modo que, igual que las direcciones, constituye un valor del escenario y no un dato del que la solución dependa. En el modo puente la dirección tampoco la fija la VM, sino que la ofrece el servidor DHCP de la red local, de modo que no puede predecirse a partir de la configuración del escenario (Oracle Corporation, 2024e). El entorno de pruebas asigna `192.168.1.51` a VM1, `192.168.1.52` a VM2 y `192.168.1.10` al *host*. Ese plan describe únicamente el entorno y no constituye una premisa de la solución, la cual identifica la dirección en tiempo de ejecución, tal como se detalla en la Subsección 2.2. Asimismo, ambas cuentan con un disco virtual adicional (`.vdi`) que la automatización deja montado como almacenamiento de datos: `/mnt/datos` con `ext4` en la primera y `/srv/datos` con `xfs` en la segunda (Nemeth et al., 2017). La comunicación entre el *host* y cada VM se realiza con la Guest Control API, que se apoya en las Guest Additions instaladas en el invitado (Oracle Corporation, 2024c).

La Figura 1 representa el escenario mediante un diagrama de despliegue UML elaborado con PlantUML, en el que se distinguen la interfaz física del *host*, la red local en modo puente, la interfaz de cada VM, los discos de almacenamiento y el flujo general de la automatización en cinco pasos: (1) la solución enciende la VM con `VBoxManage startvm`; (2) identifica en tiempo de ejecución la dirección IP de la VM a partir de la propiedad que publican las Guest Additions, con respaldo sobre la tabla ARP de la red en puente; (3) ejecuta comandos en el invitado con `VBoxManage guestcontrol`; (4) prepara el almacenamiento mediante la secuencia de identificación del disco, particionado, creación del sistema de archivos, punto de montaje, entrada en `/etc/fstab` y montaje; y (5) verifica el resultado

con `findmnt` y `df -h`, incluida la re-ejecución idempotente sobre una VM ya configurada (Nemeth et al., 2017; Oracle Corporation, 2024b; Wikipedia Contributors, 2024). El mismo flujo se aplica sobre VM2 con otros parámetros, lo que evidencia la reutilización de la solución. Las direcciones IP y los nombres de interfaz que aparecen en la figura describen el entorno de pruebas y no son datos de entrada de la solución: la dirección la obtiene ésta por sí misma en el paso 2 y de los nombres de interfaz no hace uso alguno.

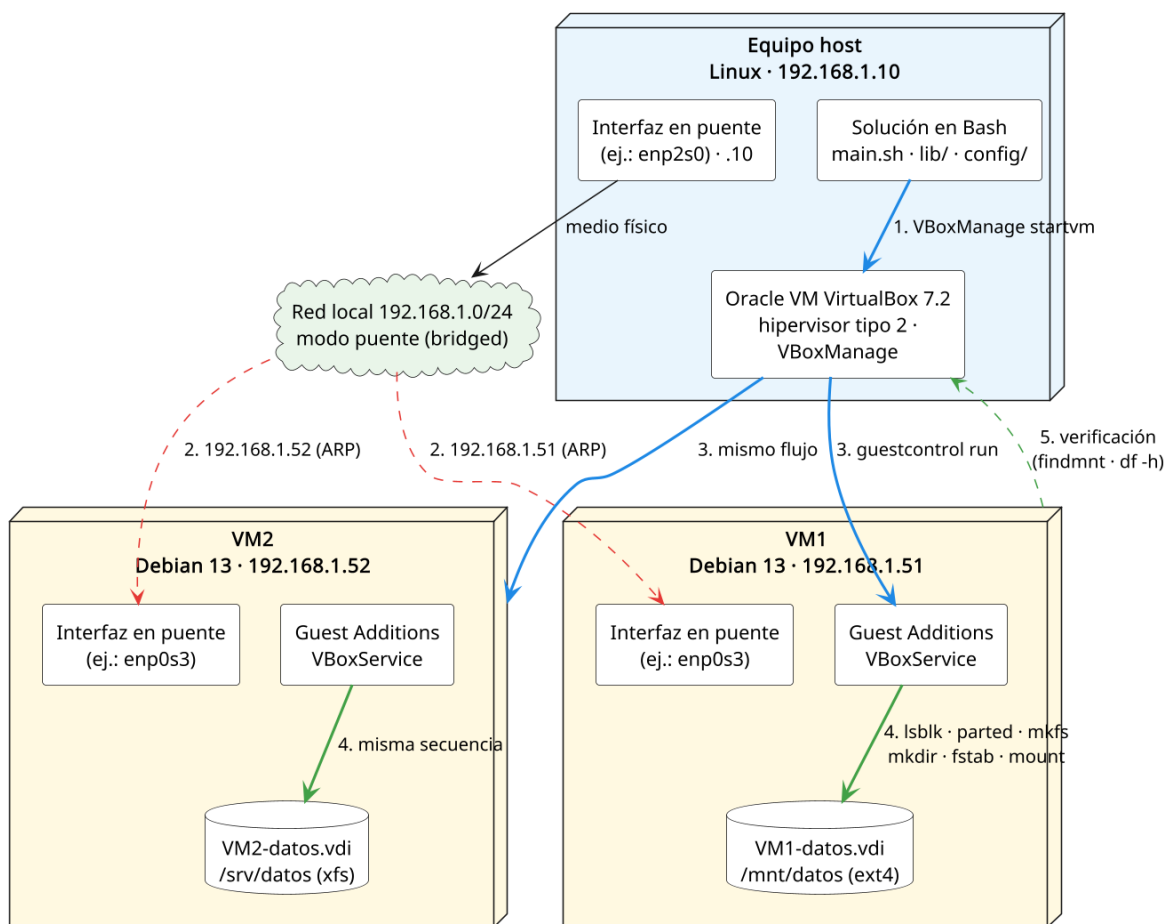


Figura 1: Escenario de despliegue: *host*, VirtualBox, red en modo puente, VM y almacenamiento adicional

## 2.2. Arquitectura y Comunicación entre Host y Guest

La solución se estructura en cuatro componentes. Un punto de entrada único, `vboxdisk`, recibe una orden (`apply`, `status` o `ld`) junto con el archivo declarativo que describe el

estado deseado y coordina la ejecución; las bibliotecas de `src/lib/` concentran funciones reutilizables de orquestación, comunicación, estado y almacenamiento; el archivo `vdisk.yml` declara, en un bloque por *VM*, los parámetros que cambian entre máquinas: disco objetivo, punto de montaje, sistema de archivos y credenciales; y el fichero de estado `state.lock` registra el último cambio realizado por el sistema sobre cada *VM* y las huellas con las que se detectan modificaciones externas (GreyCat, 2024). Con esta separación, la misma orden se aplica a *VM1* y *VM2* sin modificar el código, y las verificaciones con ShellCheck y BATS descritas en el Marco Teórico recaen sobre funciones pequeñas e independientes (Bats Contributors, 2024; Holen, 2024). El lenguaje de implementación es Bash, justificado en la Subsección 1.5 por su disponibilidad nativa en el *host* y por su acceso directo a la interfaz de línea de comandos `VBoxManage` (Oracle Corporation, 2024d).

La comunicación con el invitado se establece con la Guest Control API (`VBoxManage guestcontrol`), mecanismo que satisface el requerimiento de utilizar las Guest Additions para la interacción (Oracle Corporation, 2024c). En este mecanismo el *host* entrega las órdenes al hipervisor, éste las transporta al servicio `VBoxService` en ejecución dentro de la *VM* y el resultado regresa por el mismo canal; ese trayecto no atraviesa la red en puente, de modo que la ejecución de comandos no depende de conocer de antemano la dirección IP del invitado (Oracle Corporation, 2024a, 2024b). La autenticación se realiza con un usuario y una contraseña del propio sistema invitado, leídos del archivo de configuración antes de abrir la sesión, de manera que ninguna etapa solicita credenciales, comandos ni pulsaciones al usuario (Oracle Corporation, 2024c).

La dirección IP de la *VM* sí se identifica de forma automática, porque es un requisito del proceso y se emplea para verificar la conectividad y registrar la máquina atendida. El mecanismo principal consulta la propiedad que publican las Guest Additions en el hipervisor. La orden `VBoxManage guestproperty get VM1` lee la propiedad `/VirtualBox/GuestInfo/Net/0/V4/IP` y devuelve la dirección vigente, sin sondear la red

ni exigir servicios adicionales en el invitado (Oracle Corporation, 2024b, 2024d). Como respaldo, y en particular si la propiedad aún no se ha actualizado tras el arranque, la solución consulta la tabla ARP completa del *host* con `ip neigh` y selecciona la entrada cuya dirección MAC coincide con la del adaptador de la *VM* reportada por `VBoxManage showvminfo`, operación que no exige conocer el nombre de ninguna interfaz; si éste fuera necesario, se obtiene en tiempo de ejecución con `ip route get` a partir de la dirección recién identificada. Del mismo modo, la solución nunca utiliza el nombre de la interfaz del invitado, el cual ni la Guest Control ni la consulta de propiedades requieren. En ambos casos la dirección IP es un resultado que la solución obtiene, valida y registra, nunca un valor fijo codificado en el programa (Oracle Corporation, 2024e).

### 2.3. Algoritmo de Automatización (Idempotente)

El algoritmo se organiza en tres etapas que se ejecutan de forma secuencial y, en el camino normal, sin intervención del usuario: una etapa de preparación en el *host*, una etapa de preparación del almacenamiento que se despacha al invitado con `VBoxManage guestcontrol` y una etapa de verificación y cierre que vuelve a ejecutarse en el *host*. La Figura 2 resume ese flujo en un diagrama de procesos con dos carriles, lo que permite distinguir en cada nodo qué equipo ejecuta la acción.

La etapa de preparación comienza con la verificación declarativa: el punto de entrada carga el estado deseado de `vdisk.yml` y lo contrasta con el registro de `state.lock`. Si no hay cambios pendientes y la *VM* permanece apagada, se verifica además que la huella del disco virtual, esto es, la fecha de modificación del fichero `.vdi` y la configuración reportada por `VBoxManage showvminfo`, coincida con la registrada tras la última aplicación; como un sistema invitado apagado no admite escrituras en su interior, una huella idéntica permite omitir la *VM* sin abrir sesión en ella (Wikipedia Contributors, 2024). Si la huella cambió o la *VM* está encendida, la solución realiza un sondeo de solo lectura en el invitado y lo compara con el registro: una discrepancia atribuible a una modificación externa detiene

la operación sobre esa *VM* y solicita confirmación antes de forzar la sincronización con `vdisk.yml`, por el riesgo de pérdida de datos, de manera que la corrección también puede hacerse manualmente. Estas confirmaciones se limitan a los caminos peligrosos, como la deriva detectada o el ciclo de encendido necesario para adjuntar un disco que falta, de modo que la ejecución habitual permanece desatendida (Oracle Corporation, 2024d).

Superada la verificación declarativa, el punto de entrada comprueba si la *VM* está encendida y, de no estarlo, la enciende con `VBoxManage startvm` y aguarda la respuesta de `VBoxService`; después identifica su dirección IP en tiempo de ejecución, según se describió en la Subsección 2.2, y despacha las órdenes al invitado (Oracle Corporation, 2024b, 2024c).

La etapa de almacenamiento es idempotente porque cada acción está precedida por una guarda que consulta el estado actual antes de modificarlo (Wikipedia Contributors, 2024). La primera guarda utiliza `lsblk` para identificar el disco adicional por su tamaño y comprobar si carece de tabla de particiones; solo en ese caso crea una tabla GPT con una partición mediante `parted` (Nemeth et al., 2017). La segunda verifica, con `blkid`, si la partición ya contiene un sistema de archivos; de lo contrario ejecuta `mkfs.ext4` o `mkfs.xfs` según la configuración de la *VM*. La tercera consulta `findmnt` y el contenido de `/etc/fstab` y, únicamente si el montaje no está declarado ni activo, obtiene el UUID, crea el punto de montaje con `mkdir -p`, añade la entrada por UUID para que el montaje persista entre reinicios y ejecuta `mount -a` (Linux Foundation, 2015; Ubuntu Community, 2023).

Finalmente, la etapa de verificación confirma con `findmnt` y `df -h` que el almacenamiento quedó montado y registra el resultado: la ejecución termina con código 0 cuando la verificación es correcta y con uno de los códigos de error de la Subsección 2.4 cuando alguna guarda o verificación falla, de modo que el estado inconsistente se detecta de inmediato y la causa queda distinguible en la salida (GreyCat, 2024). Como consecuencia de las guardas, re-ejecutar la solución sobre una *VM* ya configurada no reformatea el disco ni duplica entradas en `/etc/fstab` y deja intactos los datos persistidos; la misma

secuencia aplicada a VM1 y VM2 cambia únicamente los parámetros de entrada (Hochstein & Zadka, 2023; Wikipedia Contributors, 2024).

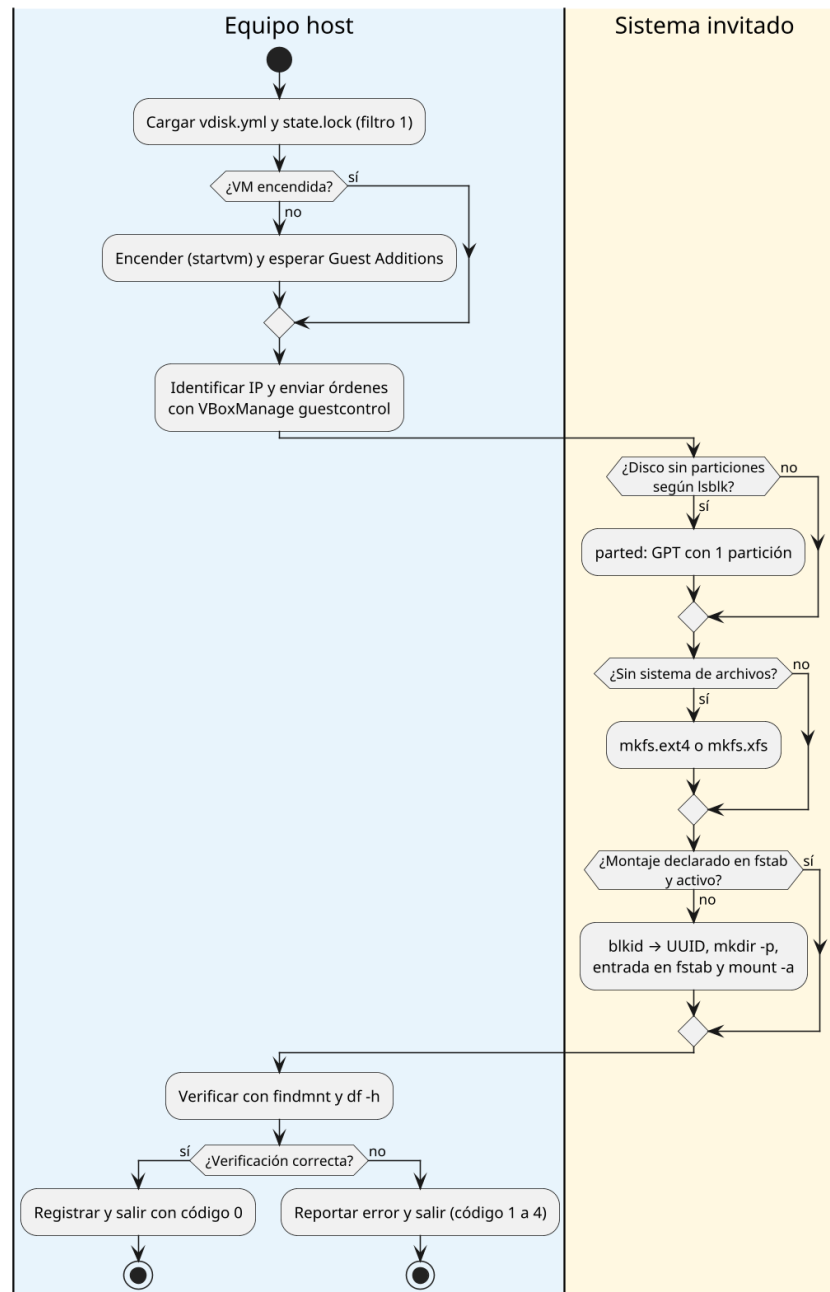


Figura 2: Diagrama de procesos de la automatización idempotente, con las etapas ejecutadas en el *host* y en el sistema invitado

## 2.4. Interfaz de la Solución

La interfaz de la solución se apoya en un enfoque declarativo inspirado en Terraform: el estado deseado se declara en un archivo y un procedimiento único de convergencia compara lo declarado con la realidad antes de actuar (HashiCorp, 2024). Ese modelo se comparte con Terraform en la separación entre lo declarado y lo real, pero no en el alcance, pues Terraform administra el estado por proyectos, mientras que esta solución trabaja a nivel global, con un único fichero de estado, `state.lock`, que comparten todas las *VM* descritas en el archivo declarativo, dado que el escenario corresponde a un solo conjunto de máquinas. Con el modelo de estado definido, la interfaz propiamente dicha se organiza en tres órdenes, de las que solo una modifica el sistema, como resume la Tabla 1.



Cuadro 1: Órdenes de la interfaz de línea de comandos

| Orden                                  | Descripción                                                                                                                                                                                                                                                      |
|----------------------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <code>vboxdisk apply</code>            | Converge todas las <i>VM</i> declaradas en <code>./vdisk.yml</code> hacia el estado deseado. Es la única orden que modifica el sistema y busca el archivo con su nombre estándar en el directorio actual; para usar otro archivo se indica con <code>-f</code> . |
| <code>vboxdisk apply -f archivo</code> | Ejecuta <code>apply</code> sobre el archivo indicado en lugar de <code>./vdisk.yml</code> .                                                                                                                                                                      |
| <code>vboxdisk apply --dry-run</code>  | Muestra el plan de cambios resultante de la verificación declarativa, sin modificar nada.                                                                                                                                                                        |
| <code>vboxdisk apply -y</code>         | Omite las confirmaciones de los caminos peligrosos para la ejecución no interactiva.                                                                                                                                                                             |
| <code>vboxdisk status</code>           | Lista las <i>VM</i> del archivo con su estado de sincronización y su dirección IP actual si están encendidas, o la última registrada, con la nota de apagada, en caso contrario.                                                                                 |
| <code>vboxdisk ld nombre</code>        | Muestra la tabla de particiones de la <i>VM</i> indicada tal como quedó registrada en <code>state.lock</code> , con la fecha del registro. Solo lee y nunca enciende la <i>VM</i> .                                                                              |
| <code>vboxdisk --help</code>           | Muestra la ayuda de la solución y termina.                                                                                                                                                                                                                       |
| <code>vboxdisk --version</code>        | Muestra la versión instalada y termina.                                                                                                                                                                                                                          |

Durante `apply` la solución informa del avance mediante una barra de progreso por etapa y dos cronómetros: el tiempo hasta que la *VM* queda disponible para recibir las órdenes de configuración, es decir, hasta que `VBoxService` responde, y el tiempo total de la ejecución, que se reporta en el resumen final. La barra, los registros y las advertencias se emiten por `stderr`, mientras que los datos, como la tabla de `status` o la salida de `ld`, salen por `stdout`, de modo que la salida de datos puede filtrarse con otros programas sin que el progreso la contamine. Cuando la salida no es un terminal, por ejemplo al redirigirla a un fichero para conservar la evidencia de una corrida, la barra se sustituye por una línea plana por etapa, de manera que el registro no contiene secuencias de control y queda listo para analizar los tiempos. En paralelo, la solución escribe esos mismos mensajes de avance y

advertencia en una bitácora por ejecución, ubicada en `$XDG_DATA_HOME/vboxdisk/state/<datetime>.log`, que conserva siempre el formato plano con independencia de si la salida es un terminal; de este modo cada corrida deja una evidencia permanente con su marca de tiempo, sin que el usuario tenga que redirigir la salida manualmente.

La solución distingue los siguientes códigos de salida:

Cuadro 2: Códigos de salida de la solución

| Código | Significado                                                                                                                                                                   |
|--------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| 0      | Todas las <i>VM</i> convergieron correctamente o no hubo cambios que aplicar.                                                                                                 |
| 1      | Error de uso o de configuración: orden desconocida, <i>VM</i> declarada que no existe en el archivo o en el hipervisor, o archivo declarativo ausente, inválido o incompleto. |
| 2      | Error de comunicación con la <i>VM</i> : <code>VBoxService</code> sin responder o dirección IP no disponible.                                                                 |
| 3      | Error de almacenamiento o de verificación: disco no identificable o sistema de archivos y montaje no disponibles.                                                             |
| 4      | Operación cancelada por el usuario en un camino peligroso.                                                                                                                    |

Cuando `apply` atiende varias *VM*, el código final refleja el error de mayor severidad: los errores globales, los códigos 1 y 4, prevalecen sobre los particulares y, entre estos, el código 3 precede al 2.

## 2.5. Sintaxis del Archivo Declarativo

El archivo `vdisk.yml` utiliza YAML, un formato de serialización legible por humanos construido sobre pares `clave: valor` (YAML Project, 2021), y se lee con la herramienta `yq`, declarada como dependencia de ejecución en la Subsección 2.6. La solución emplea únicamente tres reglas de esa sintaxis: las claves de primer nivel son los nombres de las *VM* declaradas, cada *VM* agrupa un bloque de pares `clave: valor` con sangría uniforme, y cualquier línea que comienza con `#` es un comentario. Los valores numéricos se escriben

sin comillas y las cadenas que contienen espacios o caracteres especiales se delimitan con comillas dobles. El siguiente listado muestra el archivo estándar con dos *VM* que comparten la estructura y difieren en los parámetros de almacenamiento:

Listing 1: Archivo declarativo estándar vdisk.yml

```
1 # vdisk.yml: estado deseado de las maquinas virtuales
2 VM1:
3   vm_user: operador
4   vm_pass: ChangeMe123
5   disk_file: /home/usuario/maquinas/VM1/VM1-datos.vdi
6   disk_size_mb: 4096
7   mount_point: /mnt/datos
8   fs_type: ext4
9   fs_label: datos-vm1
10
11 VM2:
12   vm_user: operador
13   vm_pass_file: /home/usuario/secretos/vm2.pass
14   disk_file: /home/usuario/maquinas/VM2/VM2-datos.vdi
15   disk_size_mb: 4096
16   mount_point: /srv/datos
17   fs_type: xfs
18   fs_label: datos-vm2
```

El bloque de cada *VM* solo puede declarar las variables reservadas que la solución reconoce, enumeradas en la Tabla 3. Cada variable define un aspecto concreto de la convergencia, y cualquier clave ajena a ese conjunto se rechaza durante la validación.

Cuadro 3: Variables reservadas del archivo declarativo

| Variable     | Obligatoria | Significado                                                                                                                       |
|--------------|-------------|-----------------------------------------------------------------------------------------------------------------------------------|
| vm_user      | Sí          | Usuario del invitado con el que Guest Control abre la sesión.                                                                     |
| vm_pass      | Sí*         | Contraseña de ese usuario, leída como valor literal.                                                                              |
| vm_pass_file | Sí*         | Ruta alternativa al fichero que contiene la contraseña.                                                                           |
| disk_file    | Sí          | Ruta del disco virtual <code>.vdi</code> en el <i>host</i> . Si no existe, <code>apply</code> lo crea y lo adjunta a la VM.       |
| disk_size_mb | Sí          | Tamaño del disco en megabytes, empleado para crearlo e identificarlo en el invitado.                                              |
| disk_device  | No          | Dispositivo en el invitado, por ejemplo <code>/dev/sdb</code> . Vacío significa autodetección por tamaño con <code>lsblk</code> . |
| mount_point  | Sí          | Punto de montaje en el invitado.                                                                                                  |
| fs_type      | Sí          | Sistema de archivos, <code>ext4</code> o <code>xfs</code> .                                                                       |
| fs_label     | No          | Etiqueta del sistema de archivos.                                                                                                 |

Se requiere al menos una de las variables `vm_pass` y `vm_pass_file`.

Al cargar el archivo, la solución valida que cada bloque contenga únicamente las variables de la Tabla 3, que las obligatorias estén presentes y que la VM declarada exista en el hipervisor; cualquier incumplimiento detiene la ejecución con código 1 antes de tocar el sistema. Los valores se resuelven con la siguiente precedencia: las opciones de línea de comandos, las variables de entorno con el prefijo `VBOXDISK_`, el bloque de la VM en `vdisk.yml` y, por último, los valores por defecto de la solución. Por ejemplo, la ruta de montaje de una VM se obtiene con una consulta como `yq -r '.VM1.mount_point'` `vdisk.yml`, lo que mantiene la lógica independiente de la máquina y satisface la reutilización sobre dos VM con parámetros distintos.

La ventaja de este diseño es la extensibilidad: incorporar una VM nueva al proceso consiste únicamente en añadir un bloque con sus variables reservadas en `vdisk.yml`,

sin modificar el código, la interfaz ni las pruebas de la solución, y las mismas órdenes la atienden junto con las ya declaradas. El archivo crece en línea con el número de máquinas, mientras que la lógica permanece invariable, que es precisamente lo que exige la reutilización sobre dos o más VM del enunciado.

## 2.6. Instalación y Sistema de Construcción

La solución se construye e instala con GNU Make, herramienta estándar de los sistemas GNU/Linux cuyo modelo de objetivos y dependencias permite declarar que la instalación solo procede después de verificar el entorno y pasar las pruebas (Mecklenburg, 2010). La orden `make install` encadena los objetivos `checkdeps`, `lint` y `test`: primero `checkdeps` comprueba en el *host* las dependencias en tiempo de ejecución, que son Bash, VBoxManage, yq e ip, y las de desarrollo, que son GNU Make, ShellCheck y BATS, y lista las que falten sin instalar nada; después `lint` ejecuta ShellCheck sobre el punto de entrada y las bibliotecas y `test` ejecuta las pruebas con BATS; solo si todo resulta satisfactorio se copian los ficheros. Las dependencias del invitado, como las Guest Additions, parted, util-linux y los paquetes de ext4 o xfs, no pueden verificarse desde el *host* y por eso se declaran como requisito de cada VM.

La instalación sigue las rutas de las variables de entorno XDG (Freedesktop.org, 2021) y no escribe fuera del área del usuario, de modo que no requiere privilegios:

Cuadro 4: Rutas de instalación según las variables XDG

| Componente  | Ruta                                                             | Contenido                                                                          |
|-------------|------------------------------------------------------------------|------------------------------------------------------------------------------------|
| Ejecutable  | <code>~/.local/bin/vboxdisk</code>                               | Punto de entrada copiado desde <code>src/vboxdisk</code> .                         |
| Bibliotecas | <code>\$XDG_DATA_HOME/vboxdisk/lib/</code>                       | Ficheros <code>src/lib/*.sh</code> .                                               |
| Recursos    | <code>\$XDG_DATA_HOME/vboxdisk/</code>                           | Plantilla <code>vdisk.yml.example</code> .                                         |
| Estado      | <code>\$XDG_DATA_HOME/vboxdisk/state.lock</code>                 | Fichero generado por la solución; no se instala ni se elimina.                     |
| Bitácoras   | <code>\$XDG_DATA_HOME/vboxdisk/state/&lt;datetime&gt;.log</code> | Una bitácora por ejecución de <code>apply</code> , escrita por la propia solución. |

Cuando `XDG_DATA_HOME` no está definida equivale a `~/.local/share`, que es su valor por defecto. El ejecutable localiza sus bibliotecas junto al propio fichero cuando corre desde el repositorio y en el directorio de datos cuando corre instalado, de manera que la misma orden sirve para desarrollo y después de `make install`. Además, el objetivo `install` asegura que el directorio `~/.local/bin` esté en la variable `PATH` del perfil del usuario: si la ruta no aparece en `~/.bashrc` o en `~/.zshrc`, la añade dentro de un bloque delimitado que `uninstall` vuelve a retirar, de modo que la orden `vboxdisk` queda disponible en cualquier sesión nueva sin privilegios y sin salir del área del usuario. El objetivo `uninstall` retira el ejecutable, las bibliotecas y ese bloque del `PATH`, y conserva el estado, las bitácoras y el archivo declarativo del usuario.

## 3. Implementación

### 3.1. Estructura del Código

La solución implementada se organiza en un punto de entrada único y una biblioteca de funciones que éste carga, de modo que cada responsabilidad queda confinada en un fichero y puede leerse y probarse por separado (GreyCat, 2024). El siguiente listado

muestra el árbol de ficheros tal como se versiona en el repositorio:

Listing 2: Árbol de ficheros de la solución

```
1 vboxdisk/
2 +-- src/
3 |   +-- vboxdisk          Punto de entrada: analiza las órdenes y
   despacha
4 |   +-- lib/
5 |       +-- common.sh     Registro, temporizadores, progreso y
   confirmaciones
6 |       +-- config.sh     Lectura y validación de vdisk.yml con yq
7 |       +-- state.sh      state.lock, huellas y bitácoras por
   ejecución
8 |       +-- vbox.sh       Envoltorios de VBoxManage: arranque, IP,
   guestcontrol
9 |       +-- storage.sh    Verificación declarativa y huellas en el
   host
10 |      +-- guest_ensure.sh Guardas de almacenamiento ejecutadas en
   el invitado
11 +-- tests/
12 |   +-- bin/VBoxManage    Doble de VBoxManage para las pruebas
   unitarias
13 |   +-- cli.bats          Análisis de argumentos y códigos de
   salida
14 |   +-- config.bats       Validación y precedencia de la
   configuración
15 |   +-- state.bats        Huellas, state.lock y bitácoras
16 +-- Makefile              Objetos help, checkdeps, lint, test,
   install y uninstall
```

|    |                                    |                                                 |
|----|------------------------------------|-------------------------------------------------|
| 17 | <code>+-+ vdisk.yml.example</code> | Plantilla declarativa de referencia             |
| 18 | <code>+-+ vdisk.yml</code>         | Archivo declarativo del usuario (no versionado) |

El punto de entrada `src/vboxdisk` coordina la solución: analiza la línea de comandos, localiza y carga las bibliotecas, despacha las tres órdenes descritas en la Subsección 2.4 y orquesta, en el caso de `apply`, las cinco etapas descritas en la Subsección 2.3; la lógica de dominio reside en `src/lib/`. En ese directorio, `common.sh` agrupa los servicios transversales: registro de mensajes con marca de tiempo por `stderr`, los dos cronómetros, la barra de progreso por etapa, las confirmaciones de los caminos peligrosos y la conversión de los códigos de salida de la Tabla 2. `config.sh` lee y valida `vdisk.yml` con `yq` y aplica la precedencia de valores de la Subsección 2.5. `state.sh` administra `state.lock` y las bitácoras por ejecución. `vbox.sh` concentra todas las llamadas a `VBoxManage`: consulta de la configuración de la *VM*, arranque, espera de `VBoxService`, lectura de propiedades, identificación de la dirección IP y despacho al invitado. `storage.sh` calcula las huellas del *host* y el plan de cambios de la verificación declarativa. `guest_ensure.sh` es el único fichero que se ejecuta dentro de la *VM*: se copia con Guest Control en cada corrida y aplica las tres guardas de la Subsección 2.3.

El punto de entrada localiza sus bibliotecas en dos lugares, según dónde se ejecute: cuando corre desde el repositorio busca `src/lib` junto al propio fichero y cuando corre instalado busca `$XDG_DATA_HOME/vboxdisk/lib`, lo que permite usar la misma orden en desarrollo y después de `make install` sin rutas codificadas (Freedesktop.org, 2021). El fichero `vdisk.yml.example` documenta la plantilla declarativa y `Makefile` implementa el sistema de construcción descrito en la Subsección 2.6. El directorio `tests/` contiene las pruebas unitarias de BATS, que emplean un doble de `VBoxManage` en `tests/bin` para ejecutarse sin *VM* ni privilegios (Bats Contributors, 2024; Holen, 2024).



## 3.2. Script / Programa Principal

El punto de entrada abre con el modo estricto de Bash y con una trampa (`trap`) que destruye los ficheros temporales de credenciales tanto en el *host* como en el invitado, con independencia de cómo termine la ejecución. A continuación analiza la línea de comandos: exige una orden de las tres válidas, acepta las banderas globales `-f`, `--dry-run` y `-y`, y responde a `--help` y `--version` sin requerir una orden. Una orden desconocida o una bandera inválida terminan con el mensaje de uso y el código 1, de modo que la solución nunca adivina lo que el usuario pretendía (GreyCat, 2024).

Listing 3: Punto de entrada: análisis de argumentos y despacho de órdenes

```
1  CMD=""
2  VBOXDISK_FILE="./vdisk.yml"
3  DRY_RUN=0
4  VBOXDISK_ASSUME_YES=0
5  LD_NAME=""
6
7  while (($#)); do
8      case "$1" in
9          apply | status | ld)
10             if [[ -n "$CMD" ]]; then
11                 die_cfg "solo se admite una orden por corrida"
12             fi
13             CMD="$1"
14             ;;
15         -f | --file)
16             if (($# < 2)); then
17                 die_cfg "la opcion $1 requiere un argumento"
18             fi
19             VBOXDISK_FILE="$2"
```

```

20         shift
21     ;;
22     --dry-run)
23         DRY_RUN=1
24     ;;
25     -y | --yes)
26         VBOXDISK_ASSUME_YES=1
27     ;;
28     -h | --help)
29         usage
30         exit 0
31     ;;
32     --version)
33         say "vboxdisk $VBOXDISK_VERSION"
34         exit 0
35     ;;
36     -*)
37         die_cfg "bandera desconocida: $1"
38     ;;
39     *)
40         if [[ "$CMD" == "ld" && -z "$LD_NAME" ]]; then
41             LD_NAME="$1"
42         else
43             die_cfg "argumento inesperado: $1"
44         fi
45     ;;
46 esac
47 shift
48 done

```

```

49
50 if [[ -z "$CMD" ]]; then
51     usage >&2
52     exit "$VBOXDISK_E_CONFIG"
53 fi
54 if ((DRY_RUN == 1)) && [[ "$CMD" != "apply" ]]; then
55     die_cfg "la opcion --dry-run solo aplica a la orden apply"
56 fi
57
58 case "$CMD" in
59     apply)
60         cmd_apply
61         ;;
62     status)
63         cmd_status
64         ;;
65     ld)
66         if [[ -z "$LD_NAME" ]]; then
67             die_cfg "la orden ld requiere el nombre de una vm"
68         fi
69         cmd_ld "$LD_NAME"
70         ;;
71 esac

```

Las órdenes `status` y `ld` comparten la carga de configuración con `apply`, pero son estrictamente de solo lectura: `status` cruza las entradas de `state.lock` con el estado que reporta `showvminfo` y con la propiedad de dirección IP cuando la VM está encendida, y `ld` muestra la tabla de particiones registrada para la VM indicada con la fecha de su registro, sin encenderla ni abrirla (Oracle Corporation, 2024d).

La orden `apply` carga primero la configuración completa: enumera las *VM* declaradas con `yq`, valida cada bloque frente a la Tabla 3, resuelve la precedencia de valores y confirma la existencia de cada *VM* en el hipervisor con una consulta de solo lectura. Cualquier incumplimiento detiene la ejecución con el código 1 antes de encender, modificar o sondear máquina alguna. La validación se concentra en `config.sh`, cuyo núcleo se muestra a continuación:

Listing 4: Validación de un bloque declarativo: claves reservadas y obligatorias

```
1  cfg_validate() {
2      local file="$1"
3      if [[ ! -f "$file" || ! -r "$file" ]]; then
4          log_error "archivo declarativo ausente o ilegible: $file"
5          return 1
6      fi
7      if ! yq eval '.' "$file" >/dev/null 2>&1; then
8          log_error "archivo declarativo no es YAML valido: $file"
9          return 1
10     fi
11     local vms
12     vms="$(cfg_vms 2>/dev/null || true)"
13     if [[ -z "$vms" ]]; then
14         log_error "no hay ninguna vm declarada en $file"
15         return 1
16     fi
17     local errors=0 vm key val fs size mount dev pass pass_file
18     for vm in $vms; do
19         if [[ ! "$vm" =~ ^[A-Za-z_][A-Za-z0-9_-]*$ ]]; then
20             log_error "nombre de vm invalido en el archivo declarativo:
                $vm"
```

```

21         errors=1
22         continue
23     fi
24     while IFS= read -r key; do
25         if [[ -z "$key" ]]; then
26             continue
27         fi
28         if ! cfg_is_reserved "$key"; then
29             log_error "$vm: clave no reservada '$key' (Tabla de
30                 variables reservadas)"
31             errors=1
32         fi
33     done < <(yq -r ".$${vm}\\" | keys | .[]" "$file" 2>/dev/null ||
34         true)
35     for key in "${VBOXDISK_REQUIRED[@]}; do
36         val="$(yq -r ".$${vm}\\" ".$${key}\" // \"\" \"$file\")"
37         if [[ -z "$val" || "$val" == "null" ]]; then
38             log_error "$vm: falta la variable obligatoria '$key'"
39             errors=1
40         fi
41     done

```

Superada la validación, apply recorre las VM declaradas y aplica a cada una la secuencia de cinco etapas de la Subsección 2.3: (1) verificación declarativa contra `state.lock` y las huellas del *host*; (2) disponibilidad de la VM, que incluye su encendido con `VBoxManage startvm --type headless` y la espera de `VBoxService`; (3) identificación de la dirección IP en tiempo de ejecución; (4) despacho del script de almacenamiento al invitado con Guest Control; y (5) verificación final y registro en `state.lock`. Cada etapa avanza la barra de progreso por `stderr` y, cuando la salida no es un terminal, emite en su lugar una

línea plana con el número de etapa, el nombre y la duración, de modo que la bitácora y las redirecciones contengan marcas de tiempo legibles y no secuencias de control. El cronómetro de disponibilidad se detiene al final de la etapa 2 y el cronómetro total al cerrar la última VM, y ambos se reportan en el resumen final (Oracle Corporation, 2024d).

La comunicación con el invitado se materializa en una secuencia de llamadas sin interacción. Una crea con `VBoxManage guestcontrol mktemp` un directorio temporal en la VM; otras dos copian a ese directorio `guest_ensure.sh` y el fichero temporal de la contraseña con `copyto`; y la última ejecuta el script con `VBoxManage guestcontrol run`, indicando el usuario y la contraseña con `--passwordfile`, el directorio de trabajo con `--cwd` y un tiempo máximo con `--timeout`; la salida estándar y de error del invitado se capturan en el *host* y se reenvían a la bitácora. Como el código que devuelve `run` mezcla los códigos de salida del proceso invitado con los errores propios del servicio Guest Control, el script termina siempre con una línea centinela de la forma `VBOXDISK_EXIT=<n>`, que el *host* interpreta para decidir si la etapa fue satisfactoria (Oracle Corporation, 2024c).

La identificación de la dirección IP implementa el mecanismo de la Subsección 2.2: consulta la propiedad que publican las Guest Additions y, si aún no está disponible, recurre a la tabla ARP del *host*, emparejando la dirección MAC del adaptador reportada por `showvminfo` con la normalizada de `ip neigh`; ambos intentos se repiten con un intervalo de dos segundos hasta agotar el tiempo de espera:

Listing 5: Identificación de la dirección IP: propiedad de Guest Additions con respaldo ARP

```
1 vbox_guestproperty_ip() {
2     local vm="$1" out
3     out="$(VBoxManage guestproperty get "$vm" /VirtualBox/GuestInfo/Net
4           /0/V4/IP 2>/dev/null || true)"
5     out="${out#Value: }"
6     if [[ "$out" =~ ^[0-9]+\.[0-9]+\.[0-9]+\.[0-9]+$ ]]; then
7         printf '%s' "$out"
8     fi
9 }
```

```

7         return 0
8     fi
9     return 1
10 }
11
12 vbox_mac() {
13     local raw
14     raw="$(vbox_info "$1" macaddress1 || true)"
15     if [[ -z "$raw" ]]; then
16         return 1
17     fi
18     printf '%s' "$raw" | sed 's/./&:/g; s/:$//' | tr '[:upper:]' '[:
        lower:]'
19 }
20
21 # vbox_arp_ip <mac>: respaldo sobre la tabla ARP de la red en puente.
22 vbox_arp_ip() {
23     local mac="$1" found
24     found="$(ip neigh show 2>/dev/null |
25         awk -v m="$mac" 'tolower($0) ~ tolower(m) { print $1; exit }')"
26     if [[ -n "$found" ]]; then
27         printf '%s' "$found"
28         return 0
29     fi
30     return 1
31 }
32
33 # vbox_detect_ip <vm> <segundos>: propiedad de Guest Additions y
    respaldo ARP,

```

```

34 # reintentando cada dos segundos hasta agotar el tiempo.
35 vbox_detect_ip() {
36     local vm="$1" timeout="${2:-60}" t0 ip mac
37     t0="$(now_s)"
38     while ;; do
39         if ip="$(vbox_guestproperty_ip "$vm")"; then
40             printf '%s' "$ip"
41             return 0
42         fi
43         if mac="$(vbox_mac "$vm")" && ip="$(vbox_arp_ip "$mac")"; then
44             printf '%s' "$ip"
45             return 0
46         fi
47         if (($(now_s) - t0 >= timeout)); then
48             return 1
49         fi
50         sleep 2
51     done
52 }

```

El script `guest_ensure.sh` recibe por argumentos los parámetros de la VM y opera en dos modos: sondeo, que solo lee y devuelve el estado actual, y convergencia, que aplica las tres guardas. El script se eleva una única vez con `sudo`, leyendo la contraseña del fichero temporal copiado al invitado, y destruye ese fichero al terminar; así ninguna operación pide datos al usuario (Oracle Corporation, 2024c). El núcleo de la convergencia es el siguiente:

Listing 6: Guardas idempotentes del script invitado: tabla, sistema de archivos y montaje

```

1 converge() {

```



```

2  # Guarda 1: tabla de particiones.
3
4  collect
5
6  if [[ "$G_TABLE" == "none" ]]; then
7      echo "guest_ensure: creando tabla GPT en $G_DEV" >&2
8      parted -s "$G_DEV" mklabel gpt mkpart primary 1MiB 100% ||
9          fail "parted fallo al crear la tabla de particiones en
10             $G_DEV"
11      partprobe "$G_DEV" 2>/dev/null || true
12      udevadm settle 2>/dev/null || true
13      wait_for_partition ||
14          fail "la particion no aparecio tras parted en $G_DEV"
15      collect
16  fi
17
18  if [[ -z "$G_PART" ]]; then
19      fail "no se identifico la particion de $G_DEV"
20  fi
21
22  # Guarda 2: sistema de archivos.
23
24  collect
25
26  if [[ -z "$G_FSTYPE" ]]; then
27      echo "guest_ensure: creando $FSTYPE en $G_PART" >&2
28      if [[ "$FSTYPE" == "ext4" ]]; then
29          if [[ -n "$LABEL" ]]; then
30              mkfs.ext4 -F -L "$LABEL" "$G_PART" || fail "mkfs.ext4
31                  fallo en $G_PART"
32          else
33              mkfs.ext4 -F "$G_PART" || fail "mkfs.ext4 fallo en
34                  $G_PART"
35          fi
36      fi
37  fi

```

```

28     else
29         if [[ -n "$LABEL" ]]; then
30             mkfs.xfs -f -L "$LABEL" "$G_PART" || fail "mkfs.xfs
31                 fallo en $G_PART"
32         else
33             mkfs.xfs -f "$G_PART" || fail "mkfs.xfs fallo en
34                 $G_PART"
35         fi
36     fi
37     collect
38
39     # Guarda 3: punto de montaje y entrada persistente en /etc/fstab;
40     solo se
41     # actua si el montaje no esta ni declarado ni activo.
42     collect
43     if [[ "$G_MOUNTED" == "yes" && "$G_FSTAB" == "yes" ]]; then
44         echo "guest_ensure: montaje ya declarado y activo en $MOUNT"
45         >&2
46         return 0
47     fi
48     if [[ -z "$G_UUID" ]]; then
49         fail "la particion $G_PART no tiene UUID tras formatear"
50     fi
51     mkdir -p "$MOUNT" || fail "no se pudo crear el punto de montaje
52         $MOUNT"
53
54     ensure_fstab
55
56     mount -a || fail "mount -a fallo"

```

```

52     collect
53     if ! findmnt --mountpoint "$MOUNT" >/dev/null 2>&1; then
54         fail "$MOUNT no quedo montado tras mount -a"
55     fi
56     echo "guest_ensure: almacenamiento disponible en $MOUNT" >&2
57     df -h "$MOUNT" >&2 || true
58     return 0
59 }

```

La primera guarda identifica el disco adicional con `lsblk` por su tamaño declarado y comprueba si carece de tabla de particiones; solo entonces crea una tabla GPT con una partición con `parted` (Nemeth et al., 2017; The Linux Kernel Archives, 2024). La segunda consulta `blkid` y, si la partición no contiene sistema de archivos, ejecuta `mkfs.ext4` o `mkfs.xfs` según `fs_type`. La tercera contrasta `findmnt` y `/etc/fstab` y, únicamente si el montaje no está declarado ni activo, obtiene el UUID, crea el punto de montaje, añade la entrada por UUID y ejecuta `mount -a` (Linux Foundation, 2015; Ubuntu Community, 2023). Al finalizar, el script devuelve en pares `clave=valor` el dispositivo, la tabla, el sistema de archivos, el UUID, el estado del montaje y la entrada en `fstab`; con esa salida, el *host* contrasta los valores declarados con la verificación que el propio script ejecuta con `findmnt` y `df -h`, y solo si esa verificación es correcta actualiza `state.lock`, de modo que una ejecución fallida no queda enmascarada en la siguiente corrida (Wikipedia Contributors, 2024).

### 3.3. Parametrización para Múltiples VMs

El requerimiento de atender más de una VM se resuelve separando lo que varía de lo que no varía. El algoritmo de cinco etapas, las órdenes de `VBoxManage`, las tres guardas de almacenamiento, los códigos de salida y el formato de `state.lock` permanecen idénticos

para toda máquina; lo único que cambia son los valores del bloque declarativo de la Tabla 3 y la identidad de la VM en el hipervisor. Por esa razón ninguna función de `src/lib/` contiene el nombre de una VM ni una dirección codificada: el punto de entrada enumera las claves de primer nivel de `vdisk.yml` con `yq eval 'keys'` y, para cada una, lee los parámetros con consultas del tipo `yq -r '.VM1.mount_point'`, en las que solo cambia el nombre de la máquina en curso (YAML Project, 2021).

En el escenario descrito, VM1 y VM2 comparten el sistema invitado y la estructura del archivo declarativo y difieren en el punto de montaje (`/mnt/datos` frente a `/srv/datos`), en el sistema de archivos (`ext4` frente a `xf`s), en la etiqueta del volumen y en la procedencia de la contraseña, que en un bloque se declara como valor literal y en el otro como ruta de un fichero. El punto de entrada recorre la lista de máquinas y ejecuta la misma función de convergencia sobre cada una, de modo que incorporar una tercera VM consiste en añadir su bloque en `vdisk.yml`, sin modificar el código, la interfaz ni las pruebas (GreyCat, 2024; HashiCorp, 2024). Además, cualquier variable reservada admite sobrescribirse con una variable de entorno con el prefijo `VBOXDISK_`, según la precedencia de la Subsección 2.5, lo que permite ajustar un parámetro puntual en una corrida determinada o en las pruebas automatizadas sin editar el archivo. El estado se almacena en un único `state.lock` con una entrada por VM, de manera que la aplicación sobre VM2 no lee ni escribe el registro de VM1 (Freedesktop.org, 2021).

### 3.4. Manejo de Errores y Casos Edge

La solución presupone que cualquier operación puede fallar y que todo fallo debe ser detectable, comunicado con claridad y no destructivo. Cuatro mecanismos sostienen ese objetivo: validación antes de actuar, tiempos de espera con reintentos, confirmación en los caminos peligrosos y códigos de salida diferenciados, descritos en la Subsección 2.4 y resumidos en la Tabla 2. La Tabla 5 detalla la respuesta prevista ante cada situación excepcional.

Cuadro 5: Respuesta de la solución ante situaciones excepcionales

| Situación                                                                                              | Respuesta de la solución                                                                                                                                            |
|--------------------------------------------------------------------------------------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Orden o bandera desconocida; <code>vdisk.yml</code> ausente, inválido o con claves ajenas a la Tabla 3 | Mensaje de uso o de validación y código 1, sin tocar el sistema.                                                                                                    |
| Credencial ausente o ilegible; VM declarada que no existe en el hipervisor                             | Rechazo durante la carga de la configuración, código 1.                                                                                                             |
| VBoxManage ausente en el <i>host</i>                                                                   | Detección en <code>checkdeps</code> al instalar y, en tiempo de ejecución, verificación inicial con código 1.                                                       |
| VM apagada                                                                                             | Se enciende con <code>startvm</code> y se espera a <code>VBoxService</code> reintentando cada dos segundos durante 120 segundos; sin respuesta, mensaje y código 2. |
| Dirección IP no disponible tras la propiedad de las Guest Additions y el respaldo ARP                  | Agotados los reintentos, mensaje y código 2, sin iniciar la etapa de almacenamiento.                                                                                |
| Disco <code>.vdi</code> ausente en el <i>host</i> o deriva detectada en el sondeo del invitado         | Confirmación previa o bandera <code>-y</code> ; sin confirmación, código 4 y VM intacta.                                                                            |
| Fallo de <code>parted</code> , <code>mkfs</code> o <code>mount</code> en el invitado                   | Se detiene la VM en curso, el fallo queda en la bitácora, código 3 y <code>state.lock</code> sin actualizar.                                                        |
| Salida redirigida a fichero o interrupción del proceso                                                 | Una línea plana por etapa, sin secuencias de escape; la trampa de salida destruye los ficheros temporales de credenciales.                                          |

Los tiempos de espera son finitos y configurables con variables de entorno con el prefijo `VBOXDISK_`: la espera de disponibilidad de la VM y la ejecución del script en el invitado, que se limita además con `--timeout` en `guestcontrol run`. Un tiempo agotado se comunica con su causa en `stderr`, se registra en la bitácora y termina con el código correspondiente, de modo que la solución nunca queda indefinidamente suspendida (GreyCat, 2024).

El carácter no destructivo descansa en dos reglas. Primera: ninguna operación que destruya datos, como crear una tabla de particiones o ejecutar `mkfs`, se ejecuta sin pasar antes por su guarda de solo lectura, de forma que una VM ya configurada recibe únicamente las acciones que le faltan (Wikipedia Contributors, 2024). Segunda: `state.lock` solo se

actualiza cuando la verificación final es correcta, de modo que una ejecución interrumpida o fallida deja el registro anterior vigente y la siguiente corrida vuelve a comparar contra la realidad. El modo `--dry-run` materializa este principio sin riesgo: calcula y muestra el plan de cambios de la verificación declarativa en el *host* y no enciende VM alguna ni modifica discos, estado ni archivos. Cuando `apply` atiende varias VM, el código final refleja el error de mayor severidad: los códigos globales 1 y 4 prevalecen sobre los particulares y, entre estos, el código 3 precede al 2; la validación global ocurre antes de cualquier confirmación, de modo que los dos códigos globales no compiten en una misma corrida.

### 3.5. Verificación Estática y Pruebas Unitarias

Antes de cualquier corrida contra una VM real, la solución se somete a los dos niveles automatizados que encadena `make install`, según se describió en la Subsección 2.6. El primero es estático: ShellCheck analiza el punto de entrada y las bibliotecas y reporta expansiones sin comillas, rutas no protegidas y demás defectos característicos de los programas en shell sin ejecutarlos (Holen, 2024). El segundo son pruebas unitarias con BATS: los ficheros de `tests/` montan en la ruta de búsqueda un doble de `VBoxManage` que responde con salidas fijas, de modo que las funciones se ejercitan sin hipervisor, sin privilegios y en milisegundos (Bats Contributors, 2024). Las pruebas cubren el análisis de argumentos y la conversión de códigos de salida (`cli.bats`), la validación y la precedencia de `vdisk.yml` (`config.bats`) y el cálculo de huellas, la lectura y escritura de `state.lock` y el registro de bitácoras (`state.bats`). El doble permite además reproducir los caminos de error de la Tabla 5, como una VM inexistente o una dirección IP no disponible, así como la interpretación de la salida `clave=valor` y de la centinela del script invitado, sin preparar ninguna condición de fallo en las máquinas reales.

### 3.6. Evidencias de las Pruebas de Funcionamiento

Las pruebas de funcionamiento verifican la solución contra las VM reales del escenario y corresponden a las seis condiciones exigidas en el punto 5 del enunciado. Cada corrida deja como evidencia la bitácora correspondiente, en la ruta descrita en la Tabla 4, junto con las capturas de pantalla que se adjuntan en el directorio `images/`. La Tabla 6 registra el diseño de cada prueba, con su condición inicial, su procedimiento y su resultado esperado; el resultado obtenido y la evidencia se completan con cada corrida contra el escenario.

| ID | Prueba                               | Con-<br>dición<br>inicial                                                                  | Procedimien-<br>to                                                                                     | Resultado es-<br>perado                                                                                                                                                       | Resul-<br>tado<br>obtenido | Evidencia                                                                                         |
|----|--------------------------------------|--------------------------------------------------------------------------------------------|--------------------------------------------------------------------------------------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|----------------------------|---------------------------------------------------------------------------------------------------|
| 1  | Configura-<br>ción inicial<br>de VM1 | VM1 en-<br>cendida;<br>disco<br>adicional<br>declarado<br>sin tabla<br>de parti-<br>ciones | Ejecutar<br><code>vboxdisk<br/>apply</code> en el<br><code>host</code> sobre<br><code>vdisk.yml</code> | Tabla GPT<br>con una par-<br>te] tición,<br>ext4<br>etiquetado,<br>montado en<br><code>/mnt/datos</code><br>y entrada<br>por UUID en<br><code>/etc/fstab</code> ;<br>código 0 | [Pendien-<br>te]           | Bitácora y<br>capturas<br>de <code>lsblk</code> ,<br><code>findmnt</code> y<br><code>df -h</code> |

| ID | Prueba                                         | Con-<br>dición<br>inicial                                                                 | Procedimien-<br>to                                                                         | Resultado es-<br>perado                                                                                         | Resul-<br>tado<br>obtenido | Evidencia                                                    |
|----|------------------------------------------------|-------------------------------------------------------------------------------------------|--------------------------------------------------------------------------------------------|-----------------------------------------------------------------------------------------------------------------|----------------------------|--------------------------------------------------------------|
| 2  | Configu-<br>ración de<br>VM2                   | VM2 en-<br>cendida;<br>disco<br>adicional<br>declarado<br>sin sis-<br>tema de<br>archivos | Ejecutar la<br>misma orden<br>vboxdisk<br>apply sobre<br>las dos VM                        | Mismo flu-<br>jo que la<br>prueba 1<br>aplicando los<br>parámetros de<br>VM2: xfs en<br>/srv/datos;<br>código 0 | [Pendien-<br>te]           | Bitácora y<br>capturas de<br>findmnt y<br>lsblk              |
| 3  | Disponibi-<br>lidad del<br>almacena-<br>miento | Prueba 1<br>comple-<br>tada en<br>VM1                                                     | Dentro de<br>VM1 ejecutar<br>df -h y lsblk,<br>y crear un<br>archivo de<br>prueba          | Los 4096 MB<br>reportados por<br>df -h, escritu-<br>ra y relectura<br>del archivo sin<br>errores                | [Pendien-<br>te]           | Captura de<br>df -h y del<br>archivo crea-<br>do             |
| 4  | Persisten-<br>cia tras<br>reinicio             | VM1 con<br>el almace-<br>namiento<br>confi-<br>gurado<br>según la<br>prueba 1             | Reiniciar VM1<br>y verificar des-<br>pués del arran-<br>que, sin ejecu-<br>tar la solución | /mnt/datos<br>montado auto-<br>máticamente<br>con la entra-<br>da UUID de<br>/etc/fstab                         | [Pendien-<br>te]           | Capturas<br>de findmnt<br>y df -h<br>después del<br>reinicio |



| ID | Prueba                                                 | Con-<br>dición<br>inicial                                                           | Procedimien-<br>to                                     | Resultado es-<br>perado                                                                                                                 | Resul-<br>tado<br>obtenido | Evidencia                                                                              |
|----|--------------------------------------------------------|-------------------------------------------------------------------------------------|--------------------------------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------|----------------------------|----------------------------------------------------------------------------------------|
| 5  | Re-<br>ejecución<br>sobre VM<br>configura-<br>da       | VM1 ya<br>configu-<br>rada, con<br>archivo<br>de datos<br>persistido                | Ejecutar<br>de nuevo<br>vboxdisk<br>apply sobre<br>VM1 | Código 0 sin<br>reformatear,<br>sin duplicar<br>entradas en<br>fstab y con<br>el archivo de<br>datos intacto                            | [Pendien-<br>te]           | Bitácora de<br>la segunda<br>corrida y<br>compara-<br>ción de<br>blkid y<br>/etc/fstab |
| 6  | Condición<br>que im-<br>pide la<br>ejecución<br>normal | VM1 en-<br>cendida<br>con el<br>adaptador<br>en modo<br>puente<br>desconec-<br>tado | Ejecutar<br>vboxdisk<br>apply sobre<br>VM1             | Mensaje claro<br>de dirección IP<br>no disponible<br>al agotar los<br>reintentos,<br>código 2 y<br>almacena-<br>miento sin<br>modificar | [Pendien-<br>te]           | Bitácora de<br>la corrida y<br>captura del<br>mensaje de<br>error                      |

Cuadro 6: Diseño y resultados de las pruebas de funcionamiento con sus evidencias

### 3.7. Tiempo de Ejecución

El tiempo de ejecución se mide con los dos cronómetros que la solución imprime en cada corrida, descritos en la Subsección 2.4: el primero registra el intervalo entre el

encendido de la *VM* y el momento en que *VBoxService* responde, es decir, en que la máquina queda disponible para recibir las órdenes de configuración, y el segundo cubre la corrida completa hasta la verificación final. Ambos quedan en la bitácora con la marca de tiempo de cada etapa, por lo que varias corridas pueden compararse sin instrumentación adicional. La Tabla 7 reúne las mediciones de tres corridas consecutivas sobre *VM1*; la columna de condiciones registra el estado inicial de la *VM*, que es la principal fuente de variación, pues una corrida fría incluye el arranque completo del sistema invitado.

Cuadro 7: Mediciones del tiempo hasta la disponibilidad de la *VM* y del tiempo total

| Corrida | Condiciones              | Disponible (s) | Total (s)   | Observaciones |
|---------|--------------------------|----------------|-------------|---------------|
| 1       | VM1 apagada al iniciar   | [Pendiente]    | [Pendiente] | [Pendiente]   |
| 2       | VM1 apagada al iniciar   | [Pendiente]    | [Pendiente] | [Pendiente]   |
| 3       | VM1 encendida al iniciar | [Pendiente]    | [Pendiente] | [Pendiente]   |

## 4. Conclusiones

### 4.1. Logros Alcanzados

Se desarrolló una solución completa de automatización que prepara el almacenamiento adicional de una *VM* GNU/Linux desde el *host*, sin intervención manual alguna: la solución enciende la *VM*, espera a que el sistema invitado esté disponible, identifica su dirección IP en tiempo de ejecución y ejecuta la secuencia de identificación del disco, particionado, creación del sistema de archivos, punto de montaje y montaje persistente en */etc/fstab* (Nemeth et al., 2017; Ubuntu Community, 2023). La comunicación *host-VM* se materializa con la Guest Control API de VirtualBox, de modo que ninguna operación exige contraseñas, comandos interactivos ni pulsaciones de tecla durante la corrida (Oracle Corporation, 2024c).

El resultado es una orden única, `vboxdisk apply`, reproducible porque la misma configuración declarativa conduce al mismo estado final, y reutilizable porque la lógica no está ligada a una VM concreta: el archivo `vdisk.yml` parametriza por máquina el usuario, el disco, el tamaño, el sistema de archivos y el punto de montaje, y la misma corrida convergió sobre VM1 con ext4 en `/mnt/datos` y sobre VM2 con xfs en `/srv/datos`. El algoritmo es idempotente: re-ejecutar la solución sobre una VM ya configurada no reformatea el disco, no pierde los datos persistidos ni duplica entradas en `/etc/fstab`, porque cada acción destructiva está precedida de una guarda de solo lectura que contrasta el estado real con el declarado (Wikipedia Contributors, 2024).

## 4.2. Cumplimiento de Requisitos del Parcial

- **Comunicación *host-VM* (3.1):** Guest Control API con obtención automática de la dirección IP a partir de la propiedad que publican las Guest Additions y respaldo sobre la tabla ARP, en red en modo puente (Oracle Corporation, 2024c, 2024e).
- **Automatización completa (3.2):** cero intervención manual; las credenciales se pasan mediante un fichero temporal que se destruye al terminar, tanto en el *host* como en el invitado.
- **Preparación de almacenamiento (3.3):** particionado, creación del sistema de archivos, punto de montaje, entrada por UUID en `fstab` y montaje, ejecutados por el script invitado `guest_ensure.sh` (Linux Foundation, 2015; The Linux Kernel Archives, 2024).
- **Ejecución repetida / idempotencia (3.4):** mecanismo descrito en la Subsección 2.3 y comprobado en la prueba 5 de la Tabla 6.
- **Aplicación a múltiples VM (3.5):** probado con dos VM cuya única diferencia son los valores de `vdisk.yml`; ninguna lógica del programa depende de una máquina

particular.

- **Tiempo de ejecución (3.6):** medido con los dos cronómetros integrados; las mediciones de tres corridas se reportan en la Tabla 7.
- **Tecnología (4):** Shell script (Bash) en un *host* GNU/Linux, con `make` como sistema de construcción y ShellCheck y BATS como verificación (Bats Contributors, 2024; GreyCat, 2024; Holen, 2024).
- **Pruebas (5):** seis pruebas diseñadas con condición inicial, procedimiento, resultado esperado, resultado obtenido y evidencia, en la Tabla 6.
- **Escenario (6):** diagrama de despliegue y descripción en la Sección 2.1.

### 4.3. Problemas Encontrados y su Solución

El desarrollo presentó cuatro problemas que se resolvieron de la siguiente manera.

#### Código de salida ambiguo

La llamada `VBoxManage guestcontrol run` devuelve códigos que mezclan los del proceso invitado con los del propio servicio Guest Control, de modo que un fallo del servicio y un código de salida del script no eran distinguibles en el *host*. Se resolvió con una línea centinela `VBOXDISK_EXIT=<n>` emitida por el script invitado sobre su salida estándar: el *host* la interpreta de forma explícita y solo asigna el código 0 cuando la centinela indica convergencia correcta.

#### Limpieza de credenciales

La trampa de salida del *host* destruía primero los ficheros temporales de contraseña locales, pero el script invitado aún necesitaba el suyo para terminar de eliminarse a sí mismo en el invitado. Se reordenó la secuencia: primero se limpian los directorios temporales del invitado con Guest Control y solo después se revuelven los

ficheros del *host*, de modo que ninguna corrida, ni siquiera una interrumpida, deja credenciales en ninguno de los dos lados.

### Nombres de VM con guion

Las consultas a `yq` construían las rutas de acceso como `VM1.mount_point`, lo que interpretaba mal los nombres con guion. Se pasan las claves siempre entre comillas (`"${vm}" . ${key}`), de modo que cualquier nombre que supere la validación se lee correctamente.

### Restricciones de Guest Control

El servicio no admite la eliminación recursiva de directorios con `guestcontrol rm` y `mktemp` anuncia el directorio creado con un prefijo de texto. Se eliminan los directorios temporales con `guestcontrol run -- /bin/rm -rf` y se extrae la ruta con el prefijo indicado en la documentación de la utilidad (Oracle Corporation, 2024c).

## 4.4. Herramientas de Inteligencia Artificial Utilizadas

Se empleó un agente de inteligencia artificial basado en un modelo de lenguaje, integrado en la terminal, para las siguientes tareas. En primer lugar, como asistente de desarrollo: diseño del enfoque declarativo y del algoritmo de cinco etapas, generación del código Bash de las bibliotecas en `src/`, del `Makefile`, de los ficheros de prueba BATS y del doble de `VBoxManage`. En segundo lugar, como verificador documental: se consultó la documentación oficial y el código fuente de VirtualBox para confirmar el comportamiento exacto de `guestcontrol run`, `mktemp`, `copyto` y `rm`, lo que permitió detectar que `rm` no admite eliminación recursiva y corregir el diseño de la limpieza del invitado antes de ejecutarlo. En tercer lugar, como redactor y corrector: redacción de las secciones del presente documento en  $\text{\LaTeX}$ , seguimiento de las convenciones del trabajo y corrección mecánica de los hallazgos de ShellCheck y de las advertencias de sobreancho de la composición tipográfica. La ejecución de las pruebas contra las VM reales y la validación final de los

resultados corresponden a los autores del trabajo.

## 4.5. Limitaciones Identificadas

La solución exige que las Guest Additions estén instaladas y funcionando en el invitado, pues tanto la espera de disponibilidad como la obtención de la dirección IP dependen de ellas (Oracle Corporation, 2024b). Supone además una red en modo puente accesible desde el *host* y un único disco adicional vacío por *VM*: no aborda LVM, varios discos ni discos con contenido previo que deba conservarse. Las credenciales de las *VM* viven en `vdisk.yml` o en un fichero legible por el usuario, suficiente para un entorno de pruebas pero no para uno productivo, donde convendría un almacén de secretos.

## 4.6. Trabajo Futuro

Extensiones naturales incluyen el soporte de LVM y de varios discos por *VM*, la ejecución desde un pipeline de integración continua, la traslación de la misma idea declarativa a otras plataformas de virtualización y la sustitución de la autenticación por contraseña por claves de `ssh` cuando el entorno lo permita. En el plano operativo, la inclusión de métricas y de una bitácora estructurada facilitaría la observabilidad de corridas sobre muchas *VM* simultáneas (Hochstein & Zadka, 2023).

## Referencias

- Bats Contributors. (2024). *Bats: Bash Automated Testing System* [Framework de testing para scripts Bash: assertions, setup/teardown, TAP output]. Consultado el 2 de octubre de 2026, desde <https://github.com/bats-core/bats-core>
- Freedesktop.org. (2021). *XDG Base Directory Specification* [Variables de entorno XDG\_DATA\_HOME, XDG\_STATE\_HOME y XDG\_CONFIG\_HOME con sus directorios por defecto]. Consultado el 3 de octubre de 2026, desde <https://specifications.freedesktop.org/basedir-spec/latest/>
- GreyCat. (2024). *Bash Guide: Best Practices* [Guía canónica: quoting, arrays, funciones, manejo de errores, set -euo pipefail]. Bash Hackers Wiki. Consultado el 2 de octubre de 2026, desde <https://mywiki.woledge.org/BashGuide>
- HashiCorp. (2024). *Terraform Documentation* [Enfoque declarativo: archivo de configuración, plan de cambios y orden apply para converger el estado deseado]. HashiCorp. Consultado el 3 de octubre de 2026, desde <https://developer.hashicorp.com/terraform/docs>
- Hochstein, L., & Zadka, M. (2023). *Ansible: Up and Running* (3.<sup>a</sup> ed.) [Capítulos sobre idempotencia, módulos declarativos, mejores prácticas de automatización]. O'Reilly Media.
- Holen, V. (2024). *ShellCheck: Shell script analysis tool* [Linter estático para detectar bugs y malas prácticas en scripts Shell]. Consultado el 2 de octubre de 2026, desde <https://www.shellcheck.net/>
- IEEE and The Open Group. (2018). *IEEE Std 1003.1-2017: Shell Command Language* [Especificación POSIX del lenguaje de comandos Shell (sh), base de portabilidad]. The Open Group. Consultado el 2 de octubre de 2026, desde [https://pubs.opengroup.org/onlinepubs/9699919799/utilities/V3\\_chap02.html](https://pubs.opengroup.org/onlinepubs/9699919799/utilities/V3_chap02.html)

- Linux Foundation. (2015). *Filesystem Hierarchy Standard (FHS) 3.0* [Estándar de jerarquía de sistemas de archivos: puntos de montaje estándar (/mnt, /media, /opt, /srv)]. Linux Foundation. Consultado el 2 de octubre de 2026, desde <https://refspecs.linuxfoundation.org/fhs.shtml>
- Mecklenburg, R. (2010). *Managing Projects with GNU Make* (3.<sup>a</sup> ed.) [Modelo de objetivos y dependencias de GNU Make para verificar, probar e instalar de forma encadenada]. O'Reilly Media.
- Nemeth, E., Snyder, G., Hein, T. R., Whaley, B., & Mackin, D. (2017). *Linux Storage Management* (5.<sup>a</sup> ed.) [Capítulos sobre discos, particiones, sistemas de archivos, LVM, RAID, fstab]. Addison-Wesley Professional.
- OpenSSH Project. (2024a). *SSH Key-Based Authentication* [Generación de claves, ssh-agent, ssh-copy-id, authorized\_keys, configuración de servidor/cliente]. OpenSSH. Consultado el 2 de octubre de 2026, desde <https://www.openssh.com/manual.html>
- OpenSSH Project. (2024b). *ssh\_config: client configuration* [Opciones: StrictHostKeyChecking y ConnectTimeout]. OpenBSD. Consultado el 2 de octubre de 2026, desde [https://man.openbsd.org/ssh\\_config.5](https://man.openbsd.org/ssh_config.5)
- Oracle Corporation. (2024a). *Advanced Topics* [Incluye: Guest Control API, VBoxManage guestcontrol, autenticación, comunicación host-guest]. Oracle. Consultado el 2 de octubre de 2026, desde <https://docs.oracle.com/en/virtualization/virtualbox/7.2/user/AdvancedTopics.html>
- Oracle Corporation. (2024b). *Guest Additions* [Documentación oficial: instalación, características y uso de Guest Additions]. Oracle. Consultado el 2 de octubre de 2026, desde <https://docs.oracle.com/en/virtualization/virtualbox/7.2/user/guestadditions.html>
- Oracle Corporation. (2024c). *Guest Control: VBoxManage guestcontrol* [Referencia de comandos para ejecutar programas, copiar archivos, crear directorios en el guest]. Oracle. Consultado el 2 de octubre de 2026, desde <https://docs.oracle.com/en/virtualization/virtualbox/7.2/user/AdvancedTopics.html#guestcontrol>



Oracle Corporation. (2024d). *VBoxManage Reference* [Referencia completa de línea de comandos: control de VMs, discos, snapshots, propiedades]. Oracle. Consultado el 2 de octubre de 2026, desde <https://docs.oracle.com/en/virtualization/virtualbox/7.2/user/VBoxManageRef.html>

Oracle Corporation. (2024e). *Virtual Networking* [Modos de red: NAT, Puente (Bridged), Solo host, Red interna]. Oracle. Consultado el 2 de octubre de 2026, desde <https://docs.oracle.com/en/virtualization/virtualbox/7.2/user/Network.html>

systemd Developers. (2024). *systemd.mount: Mount unit configuration* [Unidades de montaje systemd, alternativa a /etc/fstab, dependencias y ordenamiento]. Freedesktop.org. Consultado el 2 de octubre de 2026, desde <https://www.freedesktop.org/software/systemd/man/latest/systemd.mount.html>

The Linux Kernel Archives. (2024). *Block Devices and Partitions* [Documentación del kernel: subsistema de bloques, naming de dispositivos (/dev/sd\*, /dev/vd\*, /dev/nvme\*)]. Linux Kernel Organization. Consultado el 2 de octubre de 2026, desde <https://www.kernel.org/doc/html/latest/block/index.html>

Ubuntu Community. (2023). *Using UUIDs in /etc/fstab* [Por qué usar UUID vs /dev/sdX: persistencia tras reinicios y cambios de hardware]. Ubuntu Documentation. Consultado el 2 de octubre de 2026, desde <https://help.ubuntu.com/community/UsingUUID>

Wikipedia Contributors. (2024). *Idempotence* [Definición matemática y aplicación en DevOps/automatización de sistemas]. Wikipedia. Consultado el 2 de octubre de 2026, desde <https://en.wikipedia.org/wiki/Idempotence>

YAML Project. (2021). *YAML: YAML Ain't Markup Language* [Lenguaje de serialización legible: pares clave valor, mapas con sangría y comentarios]. YAML Project. Consultado el 3 de octubre de 2026, desde <https://yaml.org/>